EnergyMonitor: monitoreo del consumo eléctrico del hogar mediante un dispositivo IoT, una arquitectura de microservicios y alertas basadas en medición real

Juan Esteban Cortés Parra y Miguel Ángel García Artunduaga

Servicio Nacional de Aprendizaje (SENA)

Tecnólogo en Análisis y Desarrollo de Software (ADSO), ficha 3145556

28 de octubre de 2026

Resumen

Este artículo describe EnergyMonitor, un sistema para saber cuánto consume cada electrodoméstico de un hogar. Avisa cuando se pasa de un límite razonable y ofrece recomendaciones de ahorro calculadas con las lecturas del propio usuario, no con promedios ajenos. En el extremo físico hay un dispositivo con un microcontrolador ESP32 y el sensor PZEM-004T V3 que publica telemetría cada 60 segundos por MQTT 3.1.1 sobre TLS. En el servidor, un API gateway y siete microservicios en Java 21 y Spring Boot se reparten el trabajo, hablan entre sí por eventos con patrón de salida transaccional sobre RabbitMQ y guardan todo en MySQL. El usuario entra por una web hecha en React 19 o por una aplicación para Android construida con Expo. Todo corre en https://energy-monitor.tech y se actualiza con integración y despliegue continuos. Al 28 de octubre de 2026 las suites automatizadas estaban en verde: 1.806 pruebas de backend, 228 de la web, 78 de la móvil y cinco recorridos de punta a punta. La trazabilidad cubría 71 de los 108 requisitos del sistema. La cobertura de código, en cambio, no se había medido. El artículo no se queda en lo que salió bien: también reporta los indicadores que no se cumplieron y las limitaciones del trabajo, para que el lector separe lo medido de lo pendiente.

Palabras clave: monitoreo del consumo eléctrico, hogares, Internet de las cosas, microservicios, alertas, integración continua

Introducción

El problema

La factura de electricidad llega a los hogares como un número total que, en la práctica, no dice casi nada. Uno puede intuir que el calentador o el aire acondicionado son los culpables, pero rara vez lo sabe con certeza, y casi nunca sabe a qué horas del día se concentra el consumo de su casa. Sin ese dato tampoco hay forma de detectar despilfarros evidentes, como un televisor encendido en una habitación vacía o un equipo viejo que gasta más de lo necesario. El resultado es una factura más alta de la que corresponde y electricidad gastada sin razón.

Las herramientas que existen cierran el problema a medias. Medir en el tablero principal devuelve el consumo agregado de toda la vivienda, no el de cada aparato. Estimar a partir de la factura entrega promedios ajenos a la casa. Y los enchufes inteligentes comerciales se quedan en un aparato por toma, sin alertas, sin historial y sin recomendaciones que convivan en un mismo sistema. En los tres casos falta lo mismo: el circuito completo que va de la medición al aviso, y del aviso a la decisión.

Motivación de la investigación

La idea central del proyecto es medir en lugar de estimar. Un sensor instalado en el propio electrodoméstico produce lecturas de tensión, corriente, potencia activa y energía acumulada que reflejan lo que ese aparato hace en esa casa; sobre esas lecturas se pueden montar alertas y recomendaciones que el usuario reconoce como suyas, no como promedios ajenos. Fischer (2008) estudió si la retroalimentación del consumo eléctrico doméstico sirve como herramienta de ahorro, y esa pregunta se convirtió en la hipótesis de diseño de EnergyMonitor. Del lado tecnológico, el Internet de las cosas aportó el andamiaje: Atzori et al. (2010) describen la integración de objetos físicos y virtuales con capacidades de identificación y conexión, y Al-Fuqaha et al. (2015) revisan los protocolos que hacen posible esa integración, entre ellos MQTT.

A la motivación de producto se sumó una formativa. El proyecto es un proyecto del programa ADSO y lo usamos como banco de pruebas para aplicar de lleno las prácticas que el programa enseña: especificación de requisitos, diseño arquitectónico, pruebas automatizadas con trazabilidad, integración continua y documentación bilingüe. De ahí nace este artículo: contar qué se construyó y con qué evidencia se sostiene cada afirmación.

Objetivos de la investigación

El objetivo general fue desarrollar un sistema de control del recurso energético que permita monitorear en tiempo real el consumo eléctrico de los hogares e identificar los electrodomésticos que generan mayor gasto, para promover un uso eficiente y responsable de la electricidad. Ese objetivo se partió en cinco objetivos específicos, cada uno con un indicador medible. La Tabla 1 los recoge junto con la cobertura que el proyecto declara en su acta de constitución, actualizada el 28 de octubre de 2026.

Tabla 1

Objetivos específicos, indicadores y cobertura declarada

Código	Objetivo	Indicador y meta	Cobertura
OBJ-01	Medir el consumo eléctrico de cada electrodoméstico del hogar	Una lectura cada 60 segundos de voltaje, corriente, potencia activa y energía acumulada por dispositivo conectado	Cubierto; dispositivo operativo
OBJ-02	Mostrar el consumo de forma clara en una aplicación web y una móvil	Resumen e historial por hogar en la web; consulta de hogares, alertas y recomendaciones en Android, en español, inglés, francés y portugués	Web en producción; móvil integrada con la API

Código	Objetivo	Indicador y meta	Cobertura
OBJ-03	Emitir alertas automáticas ante consumo inusual o excesivo	Cuatro tipos de alerta; una desconexión se detecta en un máximo de 150 segundos	Cubierto
OBJ-04	Fomentar hábitos responsables mediante recomendaciones basadas en los datos medidos	Cinco reglas de ahorro evaluadas cada hora; la misma recomendación no se repite en 7 días	Cubierto
OBJ-05	Entregar el sistema funcionando de extremo a extremo	Demostración en vivo, desde la medición de un dispositivo físico hasta una alerta o recomendación en un teléfono, sin intervención manual, el 28 de octubre de 2026	Sistema en producción; demostración prevista para la entrega; la vinculación de un medidor en producción está pendiente de evidencia

Importancia del tema

Para los hogares la importancia es práctica: saber qué aparatos consumen más permite reaccionar a tiempo ante un consumo excesivo, y el ahorro se decide con datos propios, no con intuiciones, lo que ayuda a controlar el costo de la factura. Cada kilovatio-hora que se deja de gastar sin necesidad reduce además el impacto ambiental de la generación eléctrica. Para la formación, el proyecto deja un caso completo y demostrable: desde la medición en un

electrodoméstico real hasta el aviso en el teléfono del usuario, con la documentación y las métricas necesarias para auditarlo.

El tema además implica manejar información sensible: usuarios registrados, sus hogares y su historial de consumo. Por eso desarrollamos el proyecto con referencia a la Ley 1581 de 2012, la norma colombiana de protección de datos personales (Congreso de la República de Colombia, 2012). De esa referencia salen decisiones concretas, como la eliminación de cuenta, la verificación previa del correo electrónico y la aceptación explícita de los términos al registrarse.

Trabajos relacionados

Los trabajos previos sobre consumo doméstico giran alrededor de una pregunta incómoda: si la información se devuelve al usuario, ¿cambia su comportamiento? Fischer (2008) analizó la retroalimentación del consumo eléctrico doméstico como herramienta posible de ahorro, y de ahí salió la hipótesis de diseño de nuestro sistema. La diferencia está en dónde se mide. Esos estudios trabajan predominantemente con consumo agregado o con datos de facturación; nosotros instrumentamos aparato por aparato y cerramos el ciclo con alertas y recomendaciones dentro del mismo sistema.

Con el Internet de las cosas pasó algo parecido. La encuesta de Atzori et al. (2010) sistematiza sus componentes —identificación, sensores, redes y capacidades inteligentes— y las aplicaciones que se derivan de ellos, y la de Al-Fuqaha et al. (2015) se mete de lleno en los protocolos de habilitación y sus características de desempeño. De ahí salió nuestra decisión de comunicación: MQTT, en la especificación 3.1.1 de OASIS (Banks & Gupta, 2015), por su modelo publica-suscribe ligero, que encaja con un dispositivo que publica una lectura cada 60 segundos; y TLS en el puerto 8883, para que credenciales y lecturas viajen cifradas.

En arquitectura de software no pretendimos inventar nada nuevo. De Fowler (2002) tomamos la separación entre capas y la abstracción de la persistencia con el patrón Repository,

presente en cada servicio del proyecto. De la obra de Richardson (2018) salieron la descomposición en contextos acotados y la puerta de entrada única. El patrón de salida transaccional descrito en microservices.io (Richardson, s. f.) resolvió un problema muy concreto: anunciar eventos con exactitud cuando la escritura en base de datos y el mensaje a la cola deben ocurrir de forma consistente. El sistema arrancó como monolito modular y, durante la última fase, sus módulos se extrajeron a servicios independientes sin cambiar lo que el usuario ve.

La verificación se apoyó en dos estándares internacionales. ISO/IEC 25010:2023 define el modelo de calidad de producto con el que organizamos los indicadores (ISO, 2023), y ISO/IEC/IEEE 29119-3:2021 especifica las plantillas de documentación de pruebas —plan, especificaciones e informes— que siguieron nuestros entregables de verificación (ISO/IEC/IEEE, 2021). En seguridad de API, la lista de chequeo fue la edición 2023 del OWASP API Security Top 10 (Open Web Application Security Project, 2023), y la autenticación usa tokens JSON Web Token firmados con el algoritmo RS256, conforme al RFC 7519 (Jones et al., 2015).

EnergyMonitor queda entonces en la intersección de estas líneas: instrumentación física por aparato, transporte IoT estándar, microservicios con patrones de comunicación confiables y verificación con trazabilidad de requisitos. Su aporte no es un algoritmo nuevo. Es el ciclo completo y desplegado, del sensor al teléfono, con un reporte de indicadores que incluye tanto los cumplidos como los que no se alcanzaron.

Metodología

Ejecutamos el proyecto como proyecto formativo del programa ADSO del SENA, con dos aprendices, inicio el 1 de mayo de 2026 y una fecha de entrega que no se movió: el 28 de octubre. Esa combinación —equipo de dos personas, alcance amplio, plazo inamovible— es el escenario para el que la literatura de metodologías ligeras recomienda comunicación directa y

entregables frecuentes por encima de procesos formales (Cockburn, 2004). La planificación la concretamos en cuatro fases con avances semanales y un criterio de salida explícito por fase, como resume la Tabla 2.

Tabla 2

Fases del proyecto y criterios de salida

Fase	**Periodo**	**Objetivo y criterio de salida**
1. Fundación y flujo central	1 de mayo a 30 de junio	Un usuario registrado crea o se une a un hogar, vincula un dispositivo y ve al menos una medición real registrada.
2. Visibilidad y alertas	1 de julio a 31 de agosto	Un hogar que supera su límite genera una alerta visible en web y móvil y se entrega una notificación.
3. Recomendaciones e internacionalización	1 a 30 de septiembre	Un hogar con una semana de historial recibe al menos una recomendación basada en su propio consumo, y la interfaz se verifica en más de un idioma.
4. Consolidación y entrega	1 a 28 de octubre	El sistema se demuestra en vivo, desde la medición de un dispositivo físico hasta una alerta o recomendación en un teléfono.

El trabajo se organizó en torno a tres costumbres. La primera fue tratar la documentación como fuente de verdad: el proyecto produjo un acta de constitución, una especificación de requisitos funcionales y no funcionales con criterios de aceptación, una propuesta técnica, el diseño arquitectónico y de datos, un plan de pruebas, un informe de calidad y los manuales de usuario y técnico; todo se mantuvo en español y en inglés.

A la documentación se le sumó la trazabilidad. Cada requisito de la especificación lleva un código estable —RF-SEC-13, por ejemplo— y el plan de pruebas registró cuáles quedaban cubiertos por una prueba automatizada, siguiendo la lógica de documentación de pruebas de ISO/IEC/IEEE 29119-3:2021 (ISO/IEC/IEEE, 2021). Al 28 de octubre la especificación reunía 108 requisitos trazables.

Y por encima de todo pesó una regla de reporte que gobernó el proyecto entero: una cifra solo se presentaba como resultado si alguien la había medido. Lo que no se había medido se declaraba como tal. Así, la cobertura de código y la disponibilidad en producción aparecen explícitamente como no medidas en lugar de estimarse, y los apartados de la documentación con información pendiente se marcaron con el rótulo [POR CONFIRMAR] en vez de rellenarse con supuestos. Esa regla explica la forma de las secciones siguientes.

La calidad se trabajó con métricas definidas desde el inicio y con integración continua: cada repositorio ejecuta su suite de pruebas en GitHub Actions en cada solicitud de fusión, y el despliegue a producción requiere aprobación del entorno de despliegue. El análisis estático con ESLint quedó configurado tanto para la web como para la móvil. El modelo de ISO/IEC 25010:2023 (ISO, 2023) sirvió como marco para evaluar las características de producto —funcionalidad adecuada, fiabilidad, usabilidad, rendimiento y seguridad—, aunque algunas de esas características quedaron solo parcialmente instrumentadas, como se verá más adelante.

Implementación

El dispositivo de medición

El dispositivo que se instala en el electrodoméstico combina un microcontrolador ESP32 con Wi-Fi y Bluetooth (Espressif Systems, s. f.) con el medidor de energía PZEM-004T V3, y se alimenta de la red de 120 V mediante una fuente AC-DC HLK-PM03 encapsulada. El ESP32 lee el PZEM-004T por bus serie Modbus RTU a 9600 baudios —recepción en GPIO16, transmisión en GPIO17, dirección 0xF8— con el firmware del sensor en su versión 1.3.8.

Implementamos la lectura sobre la librería Arduino publicada para ese módulo (mandulaj, s. f.), cuya documentación reproduce las especificaciones del fabricante: de 80 a 260 V, de 0 a 100 A, hasta 23 kW, de 45 a 65 Hz, factor de potencia de 0 a 1, acumulación de energía hasta 9.999,99 kWh y exactitud declarada del 0,5 %. Ese último número es del fabricante: el proyecto nunca validó el sensor contra un medidor de referencia, y la sección de limitaciones vuelve sobre este punto.

El firmware publica telemetría —tensión, corriente, potencia activa y energía acumulada— cada 60 segundos por MQTT 3.1.1 (Banks & Gupta, 2015) con TLS en el puerto 8883, hacia el broker Mosquitto (Mosquitto, s. f.) instalado en el servidor de producción y autenticado con credenciales por dispositivo. El estado de conexión se anuncia cada 5 minutos; si el broker no recibe nada en 150 segundos, el sistema marca el dispositivo como desconectado y levanta la alerta correspondiente. Como protección ante caídas de red, el firmware mantiene un búfer local de 30 muestras que reenvía al recuperarse la conectividad. La vinculación del módulo a un hogar se realiza por Bluetooth Low Energy desde la aplicación web, con un portal de configuración como alternativa. En dinero, el prototipo cerró en COP 173.997 (Tabla 3), por debajo de la meta de COP 180.000 fijada en el acta del proyecto.

Tabla 3

Componentes y costo del prototipo de medición (COP)

Concepto	**Cantidad**	**Costo unitario**	**Subtotal**
ESP32 con Wi-Fi y Bluetooth	1	24.000	24.000
Medidor PZEM-004T-100A V3	1	70.000	70.000
Fuente AC-DC HLK-PM03	1	20.000	20.000
Cables jumper macho-macho de 20 cm	40	300	12.000
Cables jumper macho-hembra de 20 cm	40	300	12.000
Cable encauchetado 2x14 AWG	3	3.333	9.999

Concepto	Cantidad	Costo unitario	Subtotal
Cable calibre 14 rojo y negro	1	2.400	2.400
Conector JST XH2.54	1	1.300	1.300
Cables jumper hembra-hembra de 20 cm	6	383	2.298
Caja plástica (carcasa del prototipo)	1	20.000	20.000
Total del prototipo			**173.997**

El backend: un API gateway y siete microservicios

El backend está desplegado como un API gateway (puerto 8080) y siete microservicios independientes en Java 21 con Spring Boot 4.1.1 (Spring, s. f.): seguridad (8082), dispositivos (8084), mediciones (8085), hogares (8086), notificaciones (8087), alertas (8088) y recomendaciones (8089). Entre ellos conviven dos formas de comunicación. Para las consultas directas, una API interna; para las acciones asincrónicas, eventos sobre RabbitMQ (RabbitMQ, s. f.). Quien publica un evento lo hace mediante el patrón de salida transaccional (outbox): el evento se escribe en la misma transacción de base de datos que el cambio de negocio, y un proceso de reenvío lo entrega a la cola. De esa manera no existe ventana en la que el dato se guarde sin anunciarse, ni en la que se anuncie sin haberse guardado (Richardson, s. f.).

Por dentro, cada servicio sigue la arquitectura hexagonal: sus casos de uso definen puertos de entrada y de salida, y la tecnología concreta —persistencia, correo, criptografía, cola— se enchufa desde adaptadores intercambiables. El sistema acumula 64 puertos de salida para 64 contratos. La ventaja es de verificación y se nota en el día a día: las pruebas de aplicación ejercitan los casos de uso con dobles de prueba, sin levantar Spring ni MySQL, y cambiar una tecnología no toca una línea de dominio. El costo también es real, porque deja más interfaces y más clases que mantener; en un equipo de dos personas esa cuenta la hicimos a sabiendas, a cambio de la testabilidad. En total se documentaron 18 patrones de diseño, con énfasis en puertos y adaptadores, Observer, Strategy y Repository (Fowler, 2002).

La persistencia usa un esquema MySQL 8.4 (MySQL, s. f.) compartido por los servicios, con 37 tablas gestionadas por 63 *changesets* del sistema de migración versionado. Allí viven usuarios, hogares, dispositivos, mediciones, alertas, recomendaciones, notificaciones y sus tablas de apoyo. Sobre esa capa trabajan tres servicios que conviene desenredar. El de mediciones recibe cada lectura, la almacena una sola vez —aunque el dispositivo reintente— y la anuncia. El de alertas evalúa dos cosas: los niveles de consumo, donde los umbrales de 500, 1.500 y 3.000 W definen los estados LOW, MEDIUM, HIGH y CRITICAL, y los límites de cada hogar, de 6 kWh por día y 180 kWh por mes. El de recomendaciones corre cinco reglas de ahorro cada hora y se guarda de repetir la misma recomendación durante siete días.

La seguridad se resolvió en varios frentes y conviene detallarlos. La autenticación emite tokens JSON Web Token firmados con RS256 y con vigencia de 15 minutos (Jones et al., 2015), distinguiendo sesiones de usuario y sesiones administrativas. La autorización aplica roles sobre 71 rutas públicas y 14 rutas internas de la API, y las rutas internas no son alcanzables desde el exterior. El gateway limita las escrituras a 20 solicitudes POST por minuto por dirección IP y ruta, medida alineada con el riesgo de consumo ilimitado de recursos que la OWASP documenta (Open Web Application Security Project, 2023). El servidor web termina las conexiones TLS con certificados Let’s Encrypt y entrega cabeceras de seguridad: política de contenido estricta, X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy y Permissions-Policy. En el registro, el correo se verifica antes de activar la cuenta y los términos se aceptan de forma explícita; la eliminación de cuenta borra los datos del titular, de acuerdo con el marco de la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).

Los clientes

La aplicación web está construida en React 19 (React, s. f.) con Vite como empaquetador y cubre el ciclo completo de gestión: registro e inicio de sesión, hogares y

miembros, vinculación de dispositivos, historial y estadísticas de consumo, alertas, recomendaciones y preferencias de notificación. La aplicación móvil se dirige a Android y se construye con Expo SDK 57 sobre React Native 0.86 (Expo, s. f.; React Native, s. f.); su alcance es de consulta —hogares, estado de los dispositivos, alertas y recomendaciones— con notificaciones push. Ambas interfaces hablan español, inglés, francés y portugués, y la verificación de paridad entre idiomas es parte de la suite de pruebas de la móvil. El cliente web, además, soporta trabajo sin conexión: guarda hasta 30 lecturas en búfer local y las sincroniza al volver la conectividad.

Despliegue e infraestructura

El sistema corre en un VPS Hostinger KVM 2 con Ubuntu, orquestado con Docker Compose (Docker, s. f.): un contenedor por servicio, Nginx como servidor web y punto de entrada, el broker Mosquitto, RabbitMQ y MySQL. Los dominios energy-monitor.tech y mqtt.energy-monitor.tech apuntan al servidor —179.236.83.68 en las mediciones del 28 de octubre de 2026—. Cada repositorio tiene un flujo de GitHub Actions (GitHub, s. f.) que ejecuta la suite de pruebas en cada solicitud de fusión y, desde la rama principal y con aprobación del entorno de producción, despliega su componente mediante un script por SSH. La Tabla 4 recoge la pila tecnológica tal como quedó desplegada.

Tabla 4
Pila tecnológica desplegada

Componente	**Tecnología y versión**	**Función**
Dispositivo	ESP32 + PZEM-004T V3 (firmware 1.3.8), Modbus RTU 9600	Lectura de tensión, corriente, potencia y energía
Transporte	MQTT 3.1.1 con TLS en 8883; Mosquitto	Publicación de telemetría cada 60 segundos

Componente	Tecnología y versión	Función
Backend	Java 21, Spring Boot 4.1.1: gateway + 7 microservicios	API REST, reglas de negocio y eventos
Mensajería	RabbitMQ 4.1 con salida transaccional	Eventos entre servicios con entrega confiable
Persistencia	MySQL 8.4; 37 tablas, 63 changesets	Almacenamiento único de usuarios, mediciones y más
Aplicación web	React 19 con Vite	Gestión completa del hogar y del consumo
Aplicación móvil	Expo SDK 57 con React Native 0.86	Consulta y alertas en Android
Infraestructura	Docker Compose, Nginx, GitHub Actions	Despliegue en VPS y CI/CD con aprobación

Pruebas y Validación

Que los servicios respondieran no era suficiente para dar el sistema por terminado. Cada requisito necesitaba una evidencia repetible, y esa evidencia tenía que quedar escrita de modo que otra persona pudiera recorrer el mismo camino. Para el diseño de los casos de prueba y de su reporte seguimos la norma ISO/IEC/IEEE 29119-3 (ISO/IEC/IEEE, 2021); como marco de calidad usamos el modelo de producto de la familia SQuaRE (ISO, 2023).

Estrategia de verificación

El plan de pruebas divide la verificación en cinco niveles. El primero son las pruebas de integración de cada componente del backend, escritas en JUnit Jupiter 6.0.3 y ejecutadas con Maven Surefire durante cada construcción. El segundo y el tercero prueban la lógica de la web con Vitest 5.0.3 sobre jsdom, y la de la móvil con Jest dentro de jest-expo. El cuarto ensambla la pila completa con Docker Compose y corre cinco flujos de punta a punta mediante

scripts de Bash que combinan curl, jq, openssl y mosquitto_pub: registro de una cuenta, vinculación de un medidor, publicación de una medición por MQTT, disparo de una alerta y eliminación de la cuenta. El quinto es el análisis estático: ESLint 9.39.4 en la web, y expo lint junto con tsc --noEmit en la móvil.

El criterio de aceptación de cada cambio fue simple y estricto: no aceptábamos ninguna solicitud de fusión con una suite en rojo. La Tabla 5 resume el estado de los cinco niveles el 28 de octubre de 2026, fecha en la que ejecutamos todas las suites sobre el código desplegado.

Tabla 5

Resultados de las suites de prueba por nivel (28 de octubre de 2026)

Nivel	Herramienta	Pruebas	Resultado
Integración del backend (gateway y siete servicios)	JUnit Jupiter 6.0.3, Maven Surefire	1.806 en total: 773 de seguridad, 260 de hogares, 234 de notificaciones, 151 de mediciones, 129 de dispositivos, 119 de alertas, 71 de recomendaciones y 69 del gateway	En verde
Unidad de la aplicación web	Vitest 5.0.3 (jsdom)	228 pruebas en 38 archivos	En verde
Unidad de la aplicación móvil	Jest (jest-expo)	78 pruebas en 8 suites	En verde
Integración de punta a punta	Bash, curl, jq, openssl y mosquitto_pub	160 aserciones en 5 flujos: dispositivo (24), medición (28), alerta (36), actividad (60) y eliminación de cuenta (12)	En verde; el flujo de recomendaciones no se volvió a

Nivel	Herramienta	Pruebas	Resultado
	sobre Docker Compose local		ejecutar en la versión 1.3
Análisis estático y de tipos	ESLint 9.39.4 (web); expo lint y tsc --noEmit (móvil)	Sin conteo de pruebas	0 hallazgos

La distribución de esas 1.806 pruebas no es casual. Seguridad concentra 773, casi la mitad del total, porque ahí vive el registro con verificación de correo, la aceptación de términos, los refresh tokens de un solo uso, el control de acceso por roles y la administración de usuarios: precisamente el frente que la OWASP (2023) ubica como el más atacado en interfaces de programación. Hogares y notificaciones, con 260 y 234 pruebas, reflejan que las reglas de membresía y las preferencias de entrega son de las partes más enredadas del dominio.

El resultado del análisis estático pide una aclaración. Cero hallazgos de ESLint en dos aplicaciones habla de disciplina de equipo, no de ausencia de defectos: ESLint detecta patrones prohibidos y errores tipados, no lógica equivocada. La métrica que habría cerrado este capítulo es la cobertura de código, y no la tenemos. JaCoCo nunca se configuró; quedó registrado como deuda de calidad con prioridad alta (DT-11) y se asume aquí como limitación.

Trazabilidad de requisitos

El documento de especificación maneja 108 requisitos estables, con códigos idénticos en las versiones en español e inglés. De ellos, 71 cuentan con una prueba automatizada que los ejercita, es decir, el 65,7 %. Los 37 restantes se concentran en cuatro frentes que el proyecto conoce bien: la aplicación móvil, con 78 pruebas unitarias y todavía sin pruebas de componentes ni de pantallas; el firmware del ESP32; la integración por Bluetooth; y el envío de notificaciones push. Queda además documentado sin ejecutar el protocolo manual de la

móvil, doce pruebas numeradas MOV-01 a MOV-12. La brecha está descrita, no escondida, y la sección de limitaciones vuelve sobre ella.

Validación en el entorno de producción

Además de las suites locales, medimos el sistema ya desplegado el 28 de octubre de 2026, entre las 12:33 y las 12:34 UTC. La página de inicio respondió HTTP 200 en 10 de 10 peticiones, con un tiempo hasta el primer byte de 0,36 s en promedio cuando el servidor web ya estaba templado y de 0,41 s incluyendo el primer arranque en frío. La ruta /api/v1/consumption-levels del gateway contestó HTTP 401 en las cinco peticiones de prueba, con un promedio cercano a 0,37 s. Ese rechazo es justamente el comportamiento esperado, porque la ruta exige un JSON Web Token, y sirve como evidencia de que la API no entrega datos a quien no se autentica (Jones et al., 2015; Open Web Application Security Project, 2023).

La misma ronda revisó las defensas del borde. Nginx 1.28.3 sobre Ubuntu devolvía las cabeceras de política de contenido, X-Content-Type-Options, X-Frame-Options en modo DENY, Referrer-Policy y Permissions-Policy. El host mqtt.energy-monitor.tech aceptó la conexión TCP y completó el apretón TLS en el puerto 8883. El protocolo ICMP sí falló, pero el firewall del proveedor lo descarta de forma sistemática, así que ese fallo no indica indisponibilidad. Conviene decirlo con claridad: estas mediciones son puntuales. No sustituyen al monitoreo continuo de latencia y disponibilidad que el proyecto aún no tiene (DT-20), y tampoco validan el requisito RNF-REN-01, que exige menos de 2 segundos en consultas autenticadas con datos; medirlo habría requerido una sesión válida y carga representativa.

Resultados

Agrupamos los resultados en dos bloques: el cumplimiento de las metas cuantitativas que el proyecto fijó en su acta y en la especificación, y el estado real de la calidad de software al cierre. La Tabla 6 junta los indicadores con su meta documentada, el valor medido y el

estado. Cuando una meta no existe o está marcada como pendiente de confirmación, la tabla lo dice en lugar de inventar un número.

Tabla 6

Indicadores clave del proyecto: meta documentada frente a valor medido

Indicador	**Meta documentada**	**Valor medido**	**Estado**
Costo del prototipo de hardware	No superior a COP 180.000 (acta del 23 de septiembre de 2026)	COP 173.997 en los ítems 3 a 6 de la lista de compras	**Cumple** (96,7 % del presupuesto)
Cadencia de telemetría	Una lectura cada 60 segundos (RNF-REN-02)	Parámetro del firmware configurado a 60 s	Implementado; demostración en vivo el 23 de octubre
Detección de desconexión	Umbral de 150 segundos (RNF-DIS-02)	Ventana de 150 s implementada en el dispositivo	Implementado
Tolerancia a caídas del servidor	Cola local de 30 lecturas (RNF-DIS-03)	Cola circular de 30 muestras en el ESP32	Implementado
Autenticación de la API	Rechazo sin credencial (RNF-SEG-01)	5 de 5 peticiones con HTTP 401 (28/10/2026)	**Cumple**
Latencia de consultas	Menos de 2 s con datos autenticados (RNF-REN-01)	0,37 s en rechazos sin sesión; sin medición con datos	Parcial; medición pendiente

Indicador	Meta documentada	Valor medido	Estado
Latencia de la página de inicio	Sin meta cuantitativa	0,36 s hasta el primer byte (10 peticiones, 28/10/2026)	Medido puntualmente
Disponibilidad del servicio	Sin meta (RNF-DIS-01 marcado [POR CONFIRMAR])	Sin medición continua (DT-20)	Pendiente
Pruebas del backend	Suites en verde antes de cada fusión	1.806 pruebas en verde (28/10/2026)	**Cumple**
Pruebas de la web y la móvil	Sin cifra meta; existencia de suites automatizadas	228 (web) y 78 (móvil) en verde	**Cumple**; sin pruebas de componentes
Pruebas de punta a punta	Cinco flujos automatizados sobre la pila local	160 aserciones en verde	Parcial; un flujo sin re-ejecutar
Análisis estático	ESLint sin hallazgos	0 hallazgos en web y móvil	**Cumple**; aún no corre en CI
Trazabilidad requisito–prueba	Sin cifra meta documentada	71 de 108 requisitos (65,7 %)	37 requisitos sin cobertura
Cobertura de código	Umbral por definir (MP-02)	No medida: JaCoCo no instalado (DT-11)	No cumple

Del lado económico, el prototipo cerró en COP 173.997, con COP 6.003 de margen bajo el techo del acta. La lista de compras del sistema completo asciende a COP 8.073.997 en hardware y COP 48.000 en software, para un total de COP 57.021.997 que incluye todos los medidores, los controladores, los insumos de electrónica y los servicios contratados. La operación corre en un único servidor Hostinger KVM 2 con cuatro núcleos y 8 GB de memoria,

sin balanceadores ni réplicas: la misma decisión que permitió publicar la versión 1.0 en agosto y que hoy se documenta como riesgo asumido.

Lo que más nos convence no es una cifra aislada, sino la continuidad. Entre el 24 y el 28 de octubre de 2026 repartimos el monolito —Spring Modulith— en un gateway y siete servicios sin ventanas de caída, con los mismos nombres de dominio y con la web y el móvil apuntando a la misma dirección de siempre. Las suites del cuarto nivel, ejecutadas contra la pila completa, siguieron en verde durante y después de la migración; el despliegue de la versión 1.3 el 28 de octubre cerró con 160 aserciones de punta a punta en verde.

Discusión

Los datos permiten afirmar algo concreto: el ciclo completo del producto funciona. Un medidor de COP 173.997 toma lecturas cada minuto, las transporta cifradas por MQTT, las guarda en MySQL, las cruza con cinco reglas horarias, publica recomendaciones y dispara alertas de desconexión dentro del umbral de 150 segundos. Ese recorrido fue demostrado en vivo ante el comité evaluador el 23 de octubre y queda reproducible con los scripts de integración de punta a punta. Lo que los datos no permiten afirmar es que el sistema ahorre energía a sus usuarios: no hay estudio con usuarios reales, ni mediciones de consumo antes y después, ni encuestas de percepción. La literatura de retroalimentación doméstica (Fischer, 2008) avisa que medir por sí solo no garantiza ahorro. Este proyecto entrega la medición; el estudio de su efecto queda como trabajo futuro.

La arquitectura hexagonal tuvo una consecuencia práctica que no calculamos al principio: la testabilidad. Los 1.806 pruebas del backend corren en una máquina local sin depender de servicios en la nube, porque cada servicio expone sus casos de uso por puertos que las pruebas sustituyen a voluntad (Fowler, 2002; Richardson, 2018). Cuando llegó la descomposición en microservicios, ese ahorro se cobró: partir ocho componentes en dos semanas sin caídas habría sido impensable si cada servicio hubiera nacido acoplado a su base

de datos o a las credenciales de un entorno remoto. El mismo diseño sostiene el patrón de salida transaccional con RabbitMQ, que garantiza que una medición persistida termine publicada sin duplicados aunque el bus se caiga por el camino (Richardson, s. f.).

La deuda técnica es el otro hilo de la discusión. El proyecto documenta 25 hallazgos con su gravedad y su costo de corrección, y cuatro siguen abiertos con prioridad alta: las pruebas no herméticas (DT-07), la ausencia de cobertura medida (DT-11), los respaldos que viven en el mismo servidor que la operación (DT-19) y la falta de monitoreo de latencia y disponibilidad (DT-20). Ninguno de los cuatro impide usar el sistema; los cuatro lo limitan en un escenario adverso. Asumirlos en voz alta, con código y fecha, forma parte del mismo criterio de honestidad que lleva a escribir en este artículo que la disponibilidad no se mide y que la exactitud del sensor no se validó contra un patrón de referencia.

Frente a los trabajos revisados, la aportación está en el costo del ciclo cerrado. Las encuestas sobre el Internet de las cosas describen el campo en su conjunto (Atzori et al., 2010; Al-Fuqaha et al., 2015), y en ellas el monitoreo energético doméstico aparece como caso de uso consolidado; aquí se muestra su variante de bajo costo y de código abierto, con un microcontrolador ESP32 (Espressif Systems, s. f.) y un sensor PZEM-004T V3 cuya interfaz Modbus está soportada por una librería comunitaria (mandulaj, s. f.). La elección de MQTT 3.1.1 sobre TLS (Banks & Gupta, 2015; Mosquitto, s. f.), con autenticación de dispositivos mediante hash y tokens firmados con RS256 de corta duración (Jones et al., 2015), responde directamente a los riesgos de exposición de credenciales y de interfaces sin autenticación que la OWASP (2023) documenta para APIs.

El último punto es el equipo. Dos aprendices sostuvieron doce repositorios, una base de datos de 37 tablas, ocho servicios y dos aplicaciones cliente durante cinco meses. Crystal Clear (Cockburn, 2004) se eligió por ese tamaño exacto, y sus prácticas de entrega frecuente y revisión por pares se tradujeron en ramas dev y main, solicitudes de fusión con aprobación y

un canal de integración y despliegue continuos por repositorio (GitHub, s. f.). El costo de esa ambición se ve en los 23 minutos del integrador y en el despliegue manual por SSH desde la raíz del servidor (DT-21 y DT-25); el beneficio, en que al cierre cada componente desplegado tiene su propio pipeline y su propio historial de versiones.

Conclusiones

Resumen de hallazgos

1. EnergyMonitor alcanza el objetivo general que se propuso: medir el consumo eléctrico de cada electrodoméstico, mostrarlo en una web y una aplicación móvil, emitir alertas y recomendar acciones de ahorro. El prototipo cerró en COP 173.997, con COP 6.003 por debajo del presupuesto aprobado.

2. El sistema lleva desplegado en energy-monitor.tech desde agosto de 2026. El 28 de octubre pasó a funcionar como un gateway y siete microservicios sin apagar el servicio ni un minuto.

3. La cifra que resume el trabajo de verificación es 2.112 pruebas automatizadas — 1.806 del backend, 228 de la web y 78 de la móvil— más 160 aserciones de cinco flujos de punta a punta. Todas estaban en verde al cierre, junto con un análisis estático sin hallazgos.

4. No todo está cubierto. La trazabilidad alcanza 71 de los 108 requisitos; los 37 restantes se concentran en la aplicación móvil, el firmware, Bluetooth y notificaciones push, y están documentados.

5. En producción, las mediciones puntuales confirman que la API rechaza peticiones sin credencial, que las cabeceras de seguridad del servidor web están activas y que la página de inicio responde en torno a 0,36 s. Lo que no se mide de forma continua, en cambio, son la disponibilidad y la latencia con carga.

6. Y la deuda técnica no se ocultó: 25 hallazgos quedaron registrados con gravedad, costo y plan de corrección. Los cuatro de prioridad alta siguen abiertos y declarados.

Limitaciones

El sensor PZEM-004T V3 nunca se validó contra un medidor de referencia, y es la limitación más gorda del trabajo. La exactitud de 0,5 % que se invoca es la del fabricante, no una medida de laboratorio: el sistema informa con precisión de catálogo, no de patrón. La calidad del software, por su parte, está medida solo donde hay instrumentos. Hay conteo de pruebas y hallazgos de ESLint, pero no hay cobertura de código (DT-11), no hay análisis en SonarQube y no se ejecutaron pruebas de rendimiento ni de accesibilidad; estas últimas quedaron marcadas como pendientes en la propia especificación.

En lo operativo, el sistema corre en un solo servidor sin etapa de pruebas previa a producción, los respaldos se guardan en la misma máquina que sirve el servicio (DT-19) y la disponibilidad no se monitorea (DT-20). Las pruebas locales tampoco son herméticas: los perfiles de Maven descargan dependencias de repositorios externos y los flujos de punta a punta necesitan la pila completa de contenedores levantada a mano (DT-07). El transporte de telemetría usa QoS 0, nivel sin garantía de entrega, decisión funcional hasta hoy pero insuficiente si la red del hogar resulta hostil. Y el dispositivo no admite actualización por aire: corregir su firmware exige intervención física.

Falta, por último, evidencia de uso real. No hay usuarios en producción, ni encuestas, ni estudios de consumo antes y después de adoptar el sistema, así que ninguna afirmación de ahorro energético puede sostenerse. El protocolo manual de la aplicación móvil (MOV-01 a MOV-12) quedó sin ejecutar, las pruebas de componentes de React Native no existen todavía y el flujo de recomendaciones de punta a punta no se volvió a ejecutar tras la descomposición en servicios. El canal de integración y despliegue se midió con 23 minutos sobre el monolito de 1.131 pruebas; después de separar los servicios no se volvió a cronometrar.

Trabajos futuros

Calidad de software

Lo primero es cerrar las cuatro deudas de prioridad alta. Configurar JaCoCo en los ocho componentes del backend y fijar un umbral mínimo de cobertura cierra DT-11 y convierte la cifra de pruebas en un indicador comparable (MP-02); llevar las pruebas de integración a Testcontainers elimina la dependencia de perfiles locales y hace herméticas las suites (DT-07). En la aplicación móvil queremos sumar React Native Testing Library para probar componentes y pantallas, y ejecutar el protocolo MOV-01 a MOV-12 en un dispositivo físico. En la web, la prioridad es un análisis de accesibilidad según WCAG, hoy sin evidencia.

Operación

Los respaldos deben salir del servidor de producción hacia almacenamiento externo, con una prueba de restauración documentada (DT-19), y el sistema necesita monitoreo de latencia y disponibilidad con alertas hacia el propio equipo (DT-20). Eso además permitiría cumplir el requisito RNF-REN-01 con datos autenticados y carga representativa, algo que hoy no se puede afirmar. El ESLint de la web debe pasar a ejecutarse en el integrador como paso bloqueante, el despliegue debe dejar de usar la raíz del servidor (DT-25) y conviene explorar una etapa de pruebas intermedia entre dev y main.

Producto e investigación

En producto, la hoja de ruta ya incluye la actualización del firmware por aire, un medidor inteligente para usuarios sin Arduino, la estimación del costo en moneda local según la tarifa vigente y la aplicación para iOS, fuera de alcance hasta hoy. En investigación, la línea más prometedora es medir el impacto real: un estudio con hogares voluntarios que registre el consumo antes y después de adoptar el sistema, con encuestas de percepción, y la validación cruzada del PZEM-004T contra un medidor de referencia para convertir la exactitud de catálogo en exactitud verificada.

Apéndice A

Reproducibilidad

El sistema se reproduce desde los repositorios públicos del proyecto. Cada componente tiene una rama main de producción y una rama dev de integración; las solicitudes de fusión van primero a dev y de dev a main. La Tabla A1 lista los repositorios vigentes y el commit desplegado el 28 de octubre de 2026, día de la migración completa a microservicios. Los repositorios retirados, energy-monitor-backend (que conserva el firmware del ESP32 en su carpeta firmware/) y energy-monitor-devices, se mantienen como historial.

Tabla A1

Repositorios vigentes y commits desplegados el 28 de octubre de 2026

Componente y repositorio	Commit	Estado en producción
API gateway (en-monitor-api-gateway)	0b2c164	Desplegado
Servicio de seguridad (en-monitor-security-service)	f57c76b	Desplegado
Servicio de dispositivos (en-monitor-device-service)	06c1bdc	Desplegado
Servicio de mediciones (en-monitor-measurement-service)	8a9b544	Desplegado
Servicio de hogares (en-monitor-home-service)	f424083	Desplegado
Servicio de notificaciones (en-monitor-notification-service)	7b10398	Desplegado
Servicio de alertas (en-monitor-alert-service)	f050b56	Desplegado
Servicio de recomendaciones (en-monitor-recommendation-service)	aeeeec5	Desplegado

Componente y repositorio	Commit	Estado en producción
Aplicación web (Energy-Monitor-application)	698d4c9	Desplegado
Infraestructura (energy-monitor-infra)	ceb1543	Desplegado
Aplicación móvil (Energy-Monitor-Mobile)	894b65a en main; 1e4109b en dev	No aplica; no se despliega en el servidor

Para repetir la verificación, cada repositorio del backend ejecuta sus pruebas con mvn test dentro de su propio job del integrador; la web corre npm test con Vitest y la móvil npx jest. Los flujos de punta a punta viven en el directorio e2e/ de energy-monitor-infra y se lanzan con Bash contra una pila levantada con Docker Compose. Para repetir las mediciones de producción bastan dos herramientas de línea de comandos: curl con las opciones de temporización para la página de inicio y la API, y openssl s_client contra mqtt.energy-monitor.tech:8883 para comprobar el apretón TLS. Los valores reportados aquí corresponden al entorno público de energy-monitor.tech en la fecha ya indicada.

Referencias

Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, *17*(4), 2347–2376. https://doi.org/10.1109/COMST.2015.2444095

Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, *54*(15), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

Banks, A., & Gupta, R. (Eds.). (2015). *MQTT version 3.1.1: OASIS Standard*. OASIS Open. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

Cockburn, A. (2004). *Crystal clear: A human-powered methodology for small teams*. Addison-Wesley.

Congreso de la República de Colombia. (2012). *Ley 1581 de 2012 (17 de octubre): Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial 48587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Docker. (s. f.). *Docker documentation*. https://docs.docker.com/

ESLint. (s. f.). *ESLint documentation*. https://eslint.org/docs/latest/

Espressif Systems. (s. f.). *ESP32 series*. https://www.espressif.com/en/products/socs/esp32

Expo. (s. f.). *Expo documentation*. https://docs.expo.dev/

Fischer, C. (2008). Feedback on household electricity consumption: A tool for saving energy? *Energy Efficiency*, *1*(1), 79–104. https://doi.org/10.1007/s12053-008-9009-7

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

GitHub. (s. f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

ISO. (2023). *ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.

International Organization for Standardization. https://www.iso.org/standard/78176.html

ISO/IEC/IEEE. (2021). *ISO/IEC/IEEE 29119-3:2021: Software and systems engineering — Software testing — Part 3: Test documentation* (2.ª ed.). International Organization for Standardization. https://www.iso.org/standard/79429.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

mandulaj. (s. f.). *PZEM-004T-V30: Arduino library for the PZEM-004T V3.0 power meter*. GitHub. https://github.com/mandulaj/PZEM-004T-V30

Mosquitto. (s. f.). *Eclipse Mosquitto*. https://mosquitto.org/

MySQL. (s. f.). *MySQL 8.4 Reference Manual*. https://dev.mysql.com/doc/refman/8.4/en/

Nginx. (s. f.). *nginx documentation*. https://nginx.org/en/docs/

Open Web Application Security Project. (2023). *OWASP API Security Top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

RabbitMQ. (s. f.). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

React. (s. f.). *React documentation*. https://react.dev/

React Native. (s. f.). *React Native documentation*. https://reactnative.dev/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Richardson, C. (s. f.). *Transactional outbox*. microservices.io. https://microservices.io/patterns/data/transactional-outbox.html

Spring. (s. f.). *Spring Boot*. https://spring.io/projects/spring-boot

Vite. (s. f.). *Vite documentation*. https://vite.dev/